Design a Code-Deployment System

SystemsExpert design walkthrough

Clarifying questions

- Build + deploy, or deploy only?
- How many machines/regions; how big are binaries?
- How fast must a deploy reach every machine?
- Do we need rollbacks and canary/gradual rollouts?

Functional requirements

- Trigger a build for a commit SHA
- Store build artifacts
- Deploy a chosen build to all hosts (global) with progress
- Roll back to a previous build

Non-functional requirements

- Builds take minutes but must never be lost or run twice
- Deploy to hundreds of thousands of hosts in < ~30 min
- Highly available control plane

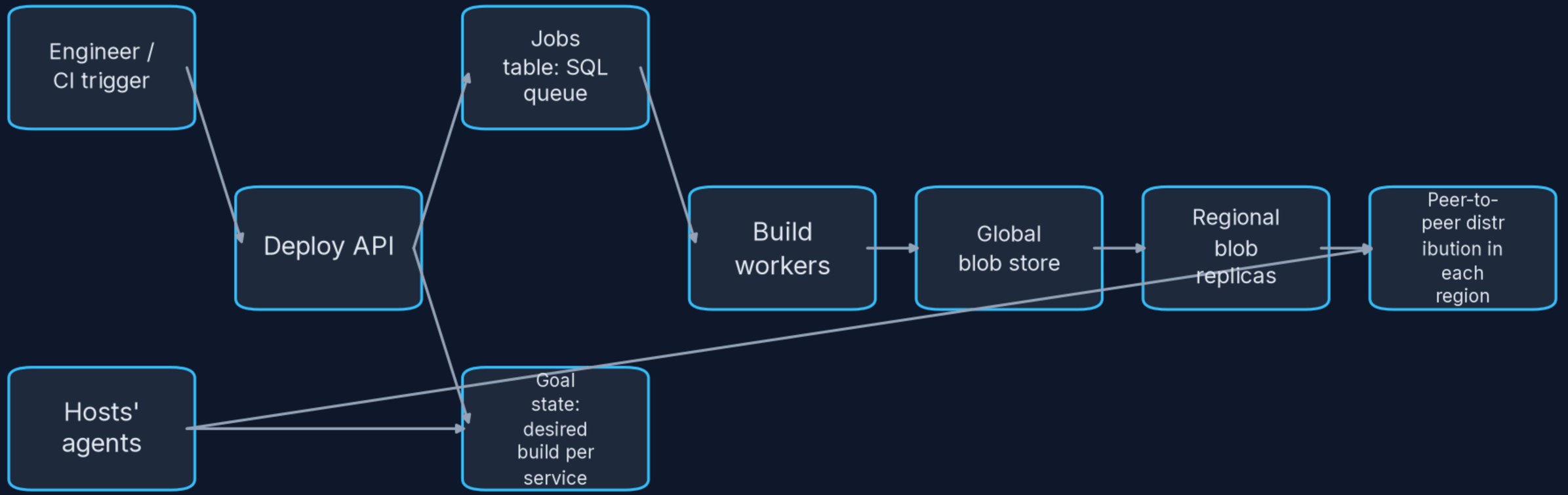
Capacity estimates

- ~thousands of builds/day; artifacts up to ~10 GB
- ~100k-1M hosts across ~10 regions
- Fan-out: 10 GB x 1M hosts is impossible from one store → regional replication + peer-to-peer distribution

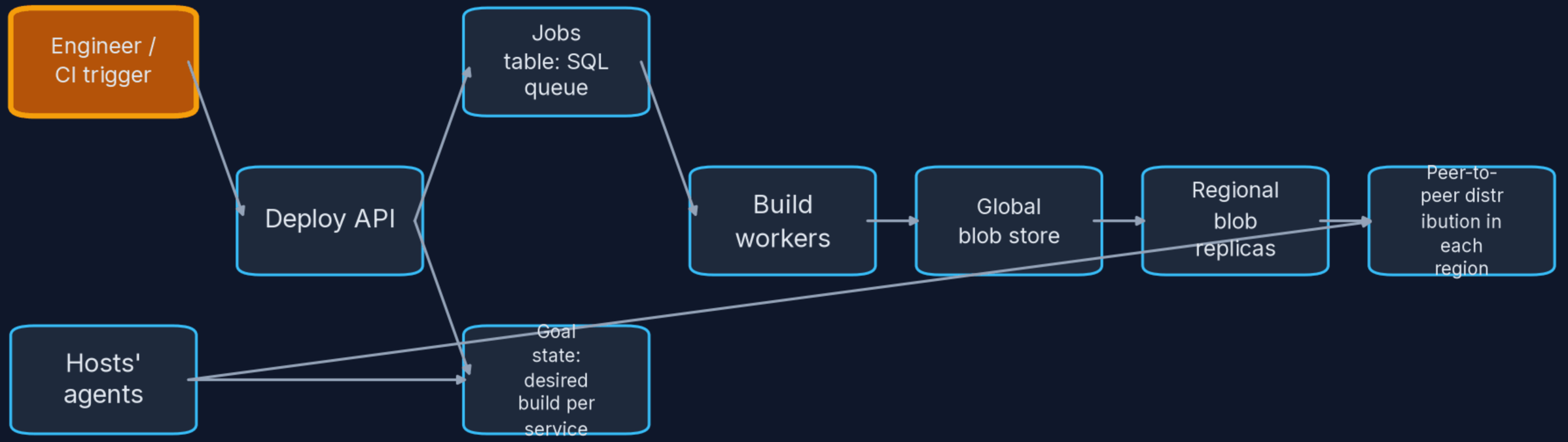
Capacity: worked out

- Artifact 10 GB × 100k hosts = 1 PB of transfer
- One store at 100 Gbps ≈ 12.5 GB/s → ~22 hours
- 10 regions + P2P: bandwidth grows with hosts
- Goal: whole fleet in < 30 min

High-level architecture

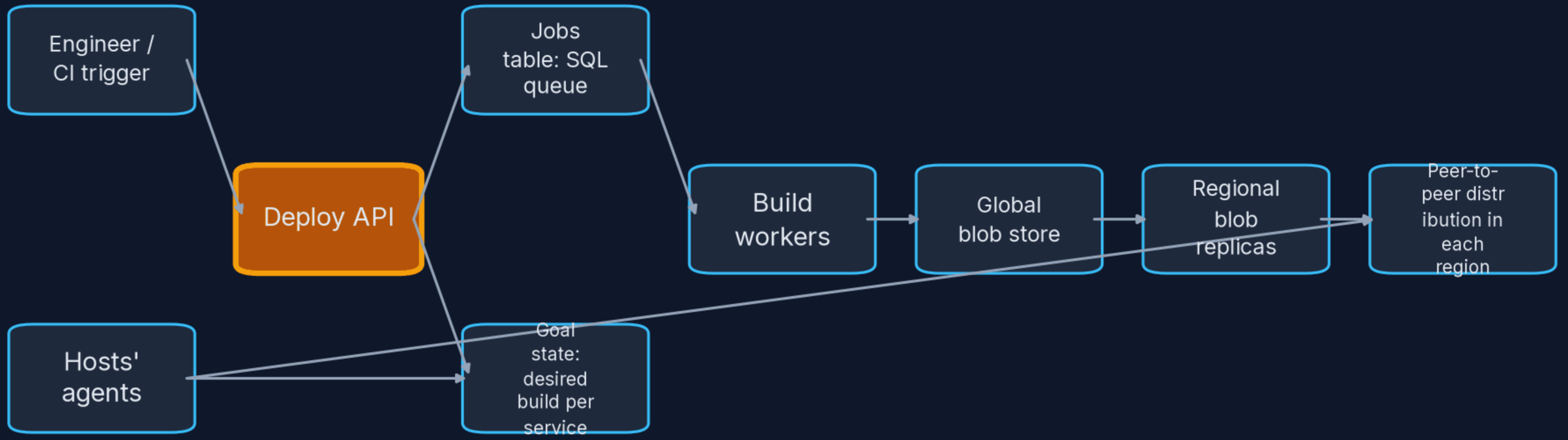


Request flow: Deploy build X



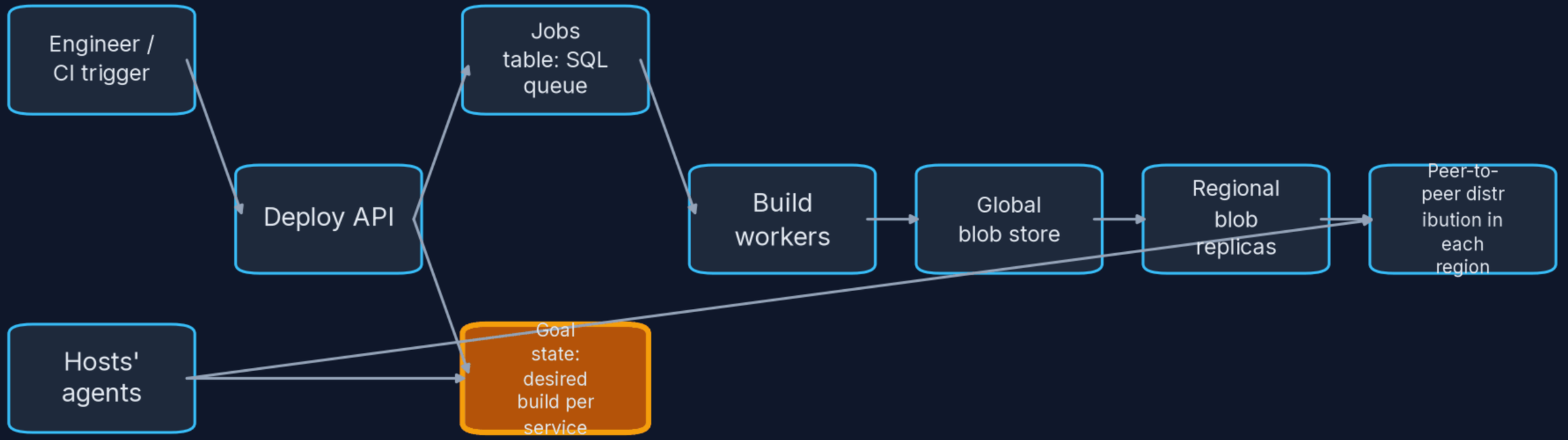
1. Engineer requests a deploy

Request flow: Deploy build X



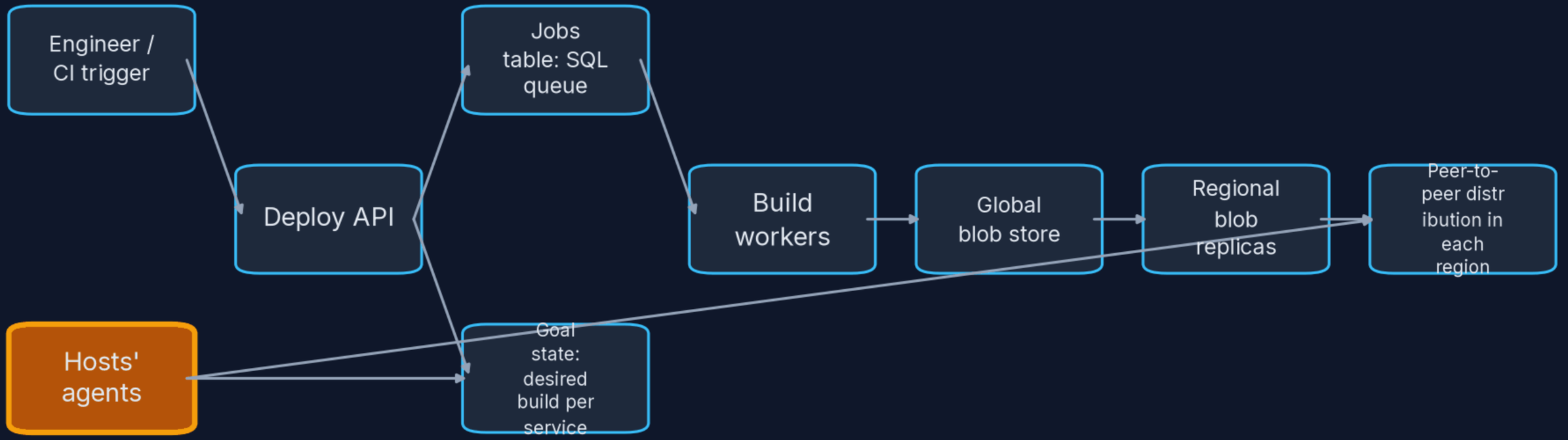
2. Deploy API records the request

Request flow: Deploy build X



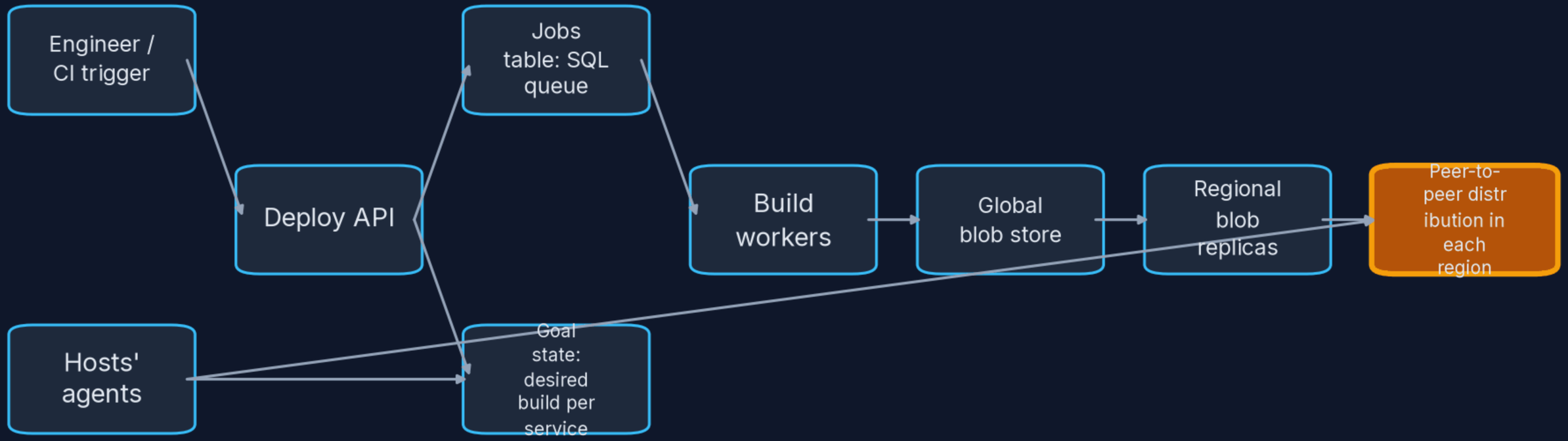
3. Goal state flips to build X

Request flow: Deploy build X



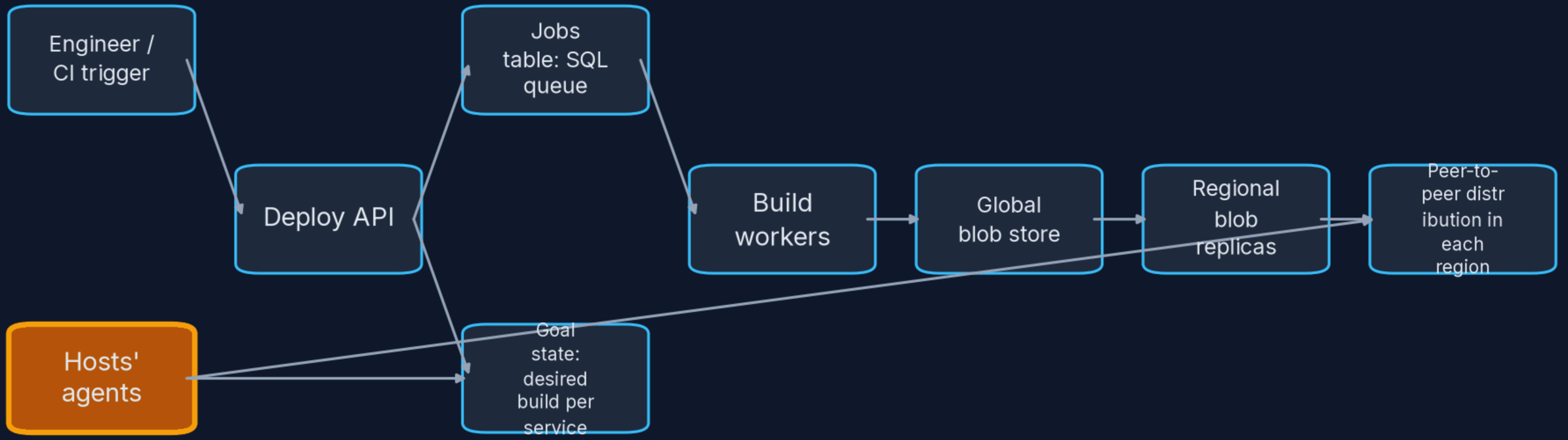
4. Agents notice the new goal

Request flow: Deploy build X



5. Hosts fetch chunks from peers

Request flow: Deploy build X



6. Swap, health-check, report

Data model

- Jobs(id, sha, status{QUEUED,RUNNING,SUCCEEDED,FAILED}, worker_id, heartbeat_at, created_at)
- Builds(sha, artifact_url, size, checksum, regions_replicated[])
- Deployments(id, service, sha, strategy, status, started_at)
- Goal_state key/value: service → sha (strongly consistent store, e.g. etcd/ZooKeeper)

API

```
POST /builds          {sha}                          -> {jobId}
GET  /builds/{sha}                                   -> status, artifact
POST /deployments     {service, sha, strategy: canary|all}
GET  /deployments/{id}                               -> per-region progress
POST /deployments/{id}/rollback
```

Queue as a SQL table

- Workers dequeue with UPDATE … SET status='RUNNING', worker_id=?
- WHERE id=(SELECT … FOR UPDATE SKIP LOCKED).
- ACID guarantees each job runs once.
- Workers heartbeat; a reaper resets stale RUNNING jobs.

Artifact distribution

- Build writes to a global store, async replicated to regional stores; deploy only proceeds once all target regions have the artifact.
- Within a region hosts fetch pieces from each other (BitTorrent-like) to avoid hot-spotting.

Goal state

- Deploy just flips the desired sha in a consistent KV store; host agents watch it, download, swap, and report health.
- Rollback = set goal to previous sha.

Key decisions

- SQL table as the build queue → exactly-once claims via row locks
- Goal state, not push commands → idempotent, easy rollback
- Regional replicas + P2P → removes the bandwidth bottleneck
- Canary stages → limits blast radius

Trade-offs & alternatives

- SQL queue vs Kafka: SQL gives exactly-once claim semantics with modest throughput, which is enough
- P2P adds complexity but removes a bandwidth bottleneck
- Canary stages slow deploys down but limit blast radius

Recap

- Clarify scope, then functional + non-functional needs
- Estimate load to size storage and fan-out
- Core pieces: Engineer / CI trigger, Deploy API, Jobs table, Build workers, Global blob store
- Know your trade-offs and failure modes